



Coding Standards Document

Client: University of Pretoria - Biophysics Group

Team Member	Student Number
Kudakwashe Mujuru	u20620111
Carlos Juma	u22602047
Nomzi Phosa	u23577160
Mbali Thobejane	u23543672

Team: CoreTech

Contact: coretech.capstone@gmail.com

Date of submission: 31/07/2026

Contents Page

Contents

1	Coding Standards	3
1.1	Variable Naming	3
1.2	File Naming	3
1.3	Functions & Classes	3
1.4	Type Casing Conventions	3
2	Project Structure	5
3	Error Handling	6
3.1	Use of Try-Except Blocks	6
3.2	Logging Errors	6
3.3	Error Propagation	6
3.4	User-Facing Error Handling	6
3.5	Error Codes	6
3.6	Validations	7
4	Testing & Debugging	8
4.1	Types of Tests	8
4.2	Code Coverage	8
5	Git Repository & Strategy	9
5.1	Git Flow	9
5.2	Branch Naming Rules	9
5.3	Commit Message Format	9
5.4	Git Hooks with Husky	10
5.4.1	Husky Configuration	10
5.4.2	Branch Name Validation	10
5.4.3	Commit Message Validation	10
5.5	Pull Request Guidelines	10
6	CI/CD	11
6.1	Linting	11
6.2	Testing in CI	11

1. Coding Standards

1.1 Variable Naming

- **camelCase** for variables and functions in TypeScript
 - Example: `userId`, `pluginConfig`, `isLoading`
- **snake_case** for Python
 - Example: `user_id`, `plugin_config`, `is_enabled`
- **UPPER_SNAKE_CASE** for constants
 - Example: `MAX_EXECUTION_TIME`, `API_BASE_URL`

1.2 File Naming

- **Python/Backend:** `snake_case`
 - Example: `plugin_service.py`, `workspace_controller.py`
- **Next.js/Frontend:**
 - Components: `PascalCase.tsx` (e.g., `PluginEditor.tsx`)
 - Services: `camelCase.ts` (e.g., `pluginService.ts`)
 - Types: `camelCase.ts` (e.g., `plugin.ts`)
- **Assets:** `icon-name.png`

1.3 Functions & Classes

- **Functions:**
 - Python: `def get_user_plugins():`
 - TypeScript: `function getPlugins()` or `const getPlugins = () =>`
- **Classes:**
 - PascalCase: `class PluginExecutionRequest`

1.4 Type Casing Conventions

To maintain consistency across the Full SMS codebase, the following casing styles must be used:

Element Type	Convention	Example
Variables (Python)	snake_case	plugin_id, user_config
Variables (TypeScript)	camelCase	pluginId, userConfig
Functions (Python)	snake_case	get_plugin_by_id()
Functions (TypeScript)	camelCase	getPluginById()
Class Names	PascalCase	PluginConfig
Constants	UPPER_SNAKE_CASE	MAX_FILE_SIZE
Database Table Names	snake_case	plugins, workspace_files
File Names (Python)	snake_case	plugin_service.py
React Components	PascalCase	PluginTable
TypeScript Interfaces	PascalCase	PluginResponse

2. Project Structure

```
Full-SMS/
├── api/ ..... FastAPI backend
│   ├── controllers/ ..... Request handlers
│   ├── services/ ..... Business logic
│   ├── routes/ ..... API endpoint definitions
│   ├── models/ ..... Pydantic models
│   ├── utils/ ..... Utility functions
│   ├── tests/ ..... Backend unit tests
│   └── main.py ..... FastAPI entry point
├── frontend/ ..... Next.js frontend
│   ├── app/ ..... Next.js app router pages
│   ├── components/ ..... Reusable UI components
│   │   ├── dashboard/ ..... Workspaces Dashboard components
│   │   ├── plugins/ ..... Plugin-related components
│   │   └── ui/ ..... Generic UI components
│   ├── contexts/ ..... React context providers
│   ├── services/ ..... API service functions
│   ├── lib/ ..... Utilities & configurations
│   ├── types/ ..... TypeScript type definitions
│   └── __tests__/ ..... Frontend unit and integration tests
├── docs/ ..... Documentation
├── .github/workflows/ ..... CI/CD configuration
├── README.md
└── requirements.txt
```

3. Error Handling

3.1 Use of Try-Except Blocks

- Used around database operations, file I/O, HDF5 parsing, and external API calls.
- Avoid wrapping large blocks, catch only relevant exceptions.

```
# Good
try:
    result = supabase.table("plugins").select("*").execute()
except Exception as e:
    raise ValueError(f"Failed to fetch plugins: {str(e)}")

# Avoid
try:
    # 50 lines of code
except:
    pass
```

3.2 Logging Errors

- Backend uses Python's logging module for error tracking.
- Frontend avoids exposing stack traces to users.
- Sensitive data (API keys, tokens) must never be logged.

3.3 Error Propagation

- Re-throw exceptions with context where necessary.
- Use custom error messages for clarity.

```
except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Plugin execution failed: {str(e)}"
    )
```

3.4 User-Facing Error Handling

- Display simple, non-technical messages in frontend (e.g., "Something went wrong").
- Use toast notifications for user feedback.
- Implement retry logic for timeouts or 5xx errors where appropriate.

3.5 Error Codes

Follow standard HTTP status codes:

Code	Usage
400	Bad Request - Invalid input
401	Unauthorized - Authentication required
403	Forbidden - Insufficient permissions
404	Not Found - Resource doesn't exist
422	Unprocessable Entity - Validation error
500	Internal Server Error

3.6 Validations

All form inputs and API payloads must be validated:

- Check for nulls, types, and logical errors.
- **Backend:** Use Pydantic schemas in FastAPI.
- **Frontend:** Use Yup with Formik for form validation.

```
# Backend validation with Pydantic
class CreatePluginRequest(BaseModel):
    name: str
    script: str
    config: PluginConfig
```

```
// Frontend validation with Yup
const validationSchema = Yup.object({
  name: Yup.string().required("Plugin name is required"),
  script: Yup.string().required("Script is required"),
});
```

4. Testing & Debugging

4.1 Types of Tests

- **Unit Tests (Backend)**
 - Tool: `pytest`
 - Files live in `api/tests/`
 - Naming: `test_<module>.py`
- **Unit Tests (Frontend)**
 - Tool: Jest with React Testing Library
 - Files live in `frontend/__tests__/`
 - Naming: `<module>.test.ts` or `<Component>.test.tsx`
- **End-to-End Testing (Planned)**
 - Tool: Cypress (partially implemented)

4.2 Code Coverage

- **Backend:** `pytest-cov`, minimum 80% target
- **Frontend:** Jest `--coverage`, minimum 80% target
- Coverage reports uploaded to Codecov via CI pipeline.

5. Git Repository & Strategy

5.1 Git Flow

We use the **Git Flow** strategy to manage parallel development safely.

Branch	Description
main	Stable production code
develop	Aggregation of completed features
feat/*	New features, always merge back to develop
bugfix/*	Bug fixes and corrections
chore/*	Maintenance dependencies, refactoring
docs/*	Documentation updates

5.2 Branch Naming Rules

- Use lowercase + hyphens
- Include ticket/issue number where applicable

Type	Pattern	Example
Feature	feat/<description>	feat/plugin-marketplace
Bug Fix	fix/<description>	fix/upload-timeout
Release	release/<description>	release/full-sms-v.1.0.0
Chore	chore/<description>	chore/update-deps
Documentation	docs/<description>	docs/api-contracts

5.3 Commit Message Format

```
<type>: <short description>
```

```
Types: feat, fix, docs, style, refactor, test, chore
```

Examples:

- feat: add plugin execution endpoint
- fix: resolve file upload timeout
- test: add workspace service tests

5.4 Git Hooks with Husky

We use **Husky** to enforce branch naming conventions and commit message formats automatically. Husky runs Git hooks that validate commits before they are accepted.

5.4.1 Husky Configuration

Husky is configured in the `.husky/` directory with the following hooks:

- **pre-commit**: Runs linting and formatting checks
- **commit-msg**: Validates commit message format against Conventional Commits
- **pre-push**: Validates branch naming convention

5.4.2 Branch Name Validation

The `pre-push` hook enforces the following branch naming pattern:

```
^(feature|bugfix|hotfix|release|chore)\/[A-Z]+-[0-9]+-[a-z0-9-]+$
```

Commits to branches not matching this pattern (except `main` and `develop`) will be rejected.

5.4.3 Commit Message Validation

The `commit-msg` hook uses **commitlint** to enforce Conventional Commits format. Invalid commit messages are rejected with an explanatory error message.

5.5 Pull Request Guidelines

- PRs must be reviewed by at least one team member
- All CI checks must pass before merging
- PRs should be linked to relevant issues/tickets
- Squash commits when merging to maintain clean history

6. CI/CD

6.1 Linting

- **Backend:**
 - Tool: `flake8`
 - Formatter: `black`
 - Config: Inline in CI workflow
- **Frontend:**
 - Tool: `eslint`
 - Config: `eslint-config-next`
- Linting is enforced via GitHub Actions on each push/PR.

6.2 Testing in CI

- **Backend:** Run all `pytest` tests with coverage reporting
- **Frontend:** Run `Jest` tests with coverage
- Both test suites are automatically triggered on PR to `develop` or `main`.
- Coverage reports uploaded to Codecov.